Agent Prompt: Marginal Ambiguity in Time-Frequency Analysis

`marginal_experiment.py`

Physics-Informed Neural Networks · Implicit Neural Representations · Cohen-Class Distributions

Contents

1 Role and Background

Role: You are a Senior Research Engineer specialising in Physics-Informed Neural Networks (PINNs) and Implicit Neural Representations (INRs) for signal processing.

Background. The *Ambiguity Problem* in time-frequency analysis refers to the fundamental non-uniqueness of any 2-D energy distribution $P(t, f)$ that is consistent with a given pair of 1-D marginals. Formally, define the marginal operator:

$$\mathcal{M}[P] = \Big(\underbrace{\textstyle\sum_f P(t,f)}_{\text{time marginal}}, \underbrace{\textstyle\sum_t P(t,f)}_{\text{freq. marginal}}\Big).$$

The operator $\mathcal{M}$ is **non-injective**: infinitely many distinct distributions P map to the same marginal pair. This means that training solely on marginal constraints is an *ill-posed inverse problem* and will generally recover a P that satisfies the data constraint perfectly while being physically meaningless.

The experiment makes this failure mode visible and quantifiable, motivating the priors introduced in follow-up scripts.

2 Objective

Create a standalone PyTorch script named `marginal_experiment.py` that:

1. Generates a windowed linear chirp signal and computes its exact 1-D marginals.
2. Trains a SIREN coordinate-MLP to satisfy those marginals, producing a learned $P(t, f)$.
3. Builds a smoothed pseudo-Wigner–Ville Distribution (pWVD) as a Cohen-class reference.
4. Produces a three-panel visualisation that makes the failure of marginal-only training *obvious*: marginals converge but the 2-D structure is wrong.
5. Prints a quantitative failure summary to `stdout`.

This is a **deliberate baseline failure case**. Do not add any regularisers, smoothness penalties, or extra priors; those belong in follow-up scripts.

3 Design Commitments

The following decisions are **fixed**. Do not introduce branches, fall-backs, or alternatives.

Design Commitment

- **Output type:** Positive energy density (Cohen-class style). Final activation is `Softplus`, so $P(t, f) \geq 0$ everywhere.
- **Normalisation:** P is divided by its sum after activation, giving a discrete PMF with $\sum_{t,f} P(t, f) = 1$.
- **Reference distribution:** Pseudo-WVD smoothed with a 2-D Gaussian kernel. This is the Cohen-class ground truth.
- **Colormap:** `viridis` throughout. No diverging colormap is needed because all values are non-negative.

- **Axis convention:** `P[t_index, f_index]`, shape `(N, N)`. Time marginal = `P.sum(dim=1)`. Freq. marginal = `P.sum(dim=0)`.

4 Step 1 — Signal Generation

4.1 Function signature

```
def generate_windowed_chirp(N=128, k=40.0, sigma=0.12, t0=0.0):
    """
    Returns
    -------
    t_grid      : Tensor (N,)  physical time axis in [-0.5, 0.5)
    x           : Tensor (N,)  complex analytic signal
    time_energy : Tensor (N,)  normalised |x(t)|^2, sums to 1
    freq_energy : Tensor (N,)  normalised |X(f)|^2, sums to 1
    """
```

Listing 1: Function signature for signal generation

4.2 Sampling and signal definition

```
t_grid = torch.linspace(-0.5, 0.5, N)          # physical axis,
    used everywhere

x = torch.exp(-(t_grid - t0)**2 / (2 * sigma**2)) \
  * torch.exp(1j * torch.pi * k * t_grid**2)    # windowed linear
      chirp
```

Listing 2: Signal construction

Rationale

A pure chirp $e^{j\pi kt^2}$ has $|x(t)|^2 = 1$ for all t, so its time marginal is flat and contains no structural information. The Gaussian envelope $e^{-(t-t_0)^2/(2\sigma^2)}$ breaks this degeneracy and makes both marginals non-trivial, which is essential for the ambiguity to be interesting rather than trivial.

4.3 Marginals

```
# --- raw (un-normalised) ----------------------------------------
raw_time = x.abs().pow(2)                       # |x(t)|^2,  shape (
    N,)

X          = torch.fft.fft(x, norm="ortho")     # unitary FFT
raw_freq   = X.abs().pow(2)                     # |X(f)|^2,  shape (
    N,)

# --- normalised to PMF ------------------------------------------
time_energy = raw_time / raw_time.sum()
freq_energy = raw_freq / raw_freq.sum()
```

Listing 3: Marginal computation

The unitary FFT (`norm="ortho"`) preserves Parseval's theorem:

$$\sum_t |x(t)|^2 = \sum_f |X(f)|^2.$$

4.4 Mandatory validations

```python
# 1. Parseval (on raw values, before normalisation)
parseval_err = (raw_time.sum() - raw_freq.sum()).abs() / raw_time.sum()
assert parseval_err < 1e-3, f"Parseval violation: {parseval_err:.2e}"

# 2. Non-negativity
assert raw_time.min() >= 0,  "time_energy has negative values"
assert raw_freq.min() >= 0,  "freq_energy has negative values"

# 3. Finiteness
assert torch.isfinite(raw_time).all(), "time_energy contains NaN/Inf"
assert torch.isfinite(raw_freq).all(), "freq_energy contains NaN/Inf"

# 4. Normalisation
assert (time_energy.sum() - 1.0).abs() < 1e-6, "time_energy does not sum to 1"
assert (freq_energy.sum() - 1.0).abs() < 1e-6, "freq_energy does not sum to 1"
```

Listing 4: Assertions in generate_windowed_chirp

5 Step 2 — Coordinate Grid

```python
N = 128

t_norm = torch.linspace(-1.0, 1.0, N)          # normalised time, [-1, 1]
f_norm = torch.linspace(-1.0, 1.0, N)          # normalised freq, [-1, 1]

TT, FF = torch.meshgrid(t_norm, f_norm, indexing="ij")
# TT shape (N, N): TT[i, j] = t_norm[i]
# FF shape (N, N): FF[i, j] = f_norm[j]

coords = torch.stack([TT.reshape(-1), FF.reshape(-1)], dim=-1)  # (N*N, 2)
```

Listing 5: Grid construction

```python
assert coords[:, 0].min().item() == -1.0 and coords[:, 0].max().item() == 1.0, \
    "t_norm out of [-1, 1]"
assert coords[:, 1].min().item() == -1.0 and coords[:, 1].max().item() == 1.0, \
    "f_norm out of [-1, 1]"
```

Listing 6: Grid validation

Move `coords` and all target tensors to `device` *before* entering the training loop.

6 Step 3 — Model Architecture (SIREN INR)

6.1 Network structure

Layer	Operation	Activation
Input	—	—
Hidden 1	`Linear(2, 256)`	$\sin(\omega_0 \cdot \mathbf{W}\mathbf{x} + \mathbf{b})$
Hidden 2	`Linear(256, 256)`	$\sin(\omega_0 \cdot \mathbf{W}\mathbf{x} + \mathbf{b})$
Hidden 3	`Linear(256, 256)`	$\sin(\omega_0 \cdot \mathbf{W}\mathbf{x} + \mathbf{b})$
Output	`Linear(256, 1)`	`Softplus`

Hyperparameter: $\omega_0 = 30$.

6.2 SIREN initialisation

Warning

Incorrect initialisation will cause SIREN to behave like a random hash function. Use the exact scheme below.

```python
import math

def init_siren_layer(linear, is_first, omega0):
    fan_in = linear.weight.size(1)
    if is_first:
        bound = 1.0 / fan_in
    else:
        bound = math.sqrt(6.0 / fan_in) / omega0
    nn.init.uniform_(linear.weight, -bound, bound)
    # bias
    bias_bound = 1.0 / math.sqrt(fan_in)
    nn.init.uniform_(linear.bias, -bias_bound, bias_bound)
```

Listing 7: SIREN init scheme

Apply `init_siren_layer` to all hidden layers. Leave the output `Linear(256, 1)` with its default PyTorch init.

6.3 Forward pass

```python
def forward(self, coords):
    # coords: (M, 2)     normalised (t, f) pairs
    out = self.net(coords)       # (M, 1)     raw pre-activation
    out = F.softplus(out)        # (M, 1)     non-negative energy
    return out.squeeze(-1)       # (M,)
```

Listing 8: TFD_Network forward

7 Step 4 — Training Loop

7.1 Device selection and reproducibility

```python
import torch, numpy as np

device = (
    "cuda" if torch.cuda.is_available() else
    "mps"  if torch.backends.mps.is_available() else
    "cpu"
)
print(f"Using device: {device}")

torch.manual_seed(42)
np.random.seed(42)
if device == "cuda":
    torch.backends.cudnn.deterministic = True
```

Listing 9: Device setup

7.2 Optimiser

```python
optimizer = torch.optim.Adam(net.parameters(), lr=1e-4)
```

No learning-rate scheduler. No early stopping. **Run exactly 2000 epochs.**

7.3 Forward pass per epoch

```python
optimizer.zero_grad()

pred_flat = net(coords)                # (N*N,)
P = pred_flat.view(N, N)               # (N, N)      axis: P[t_idx,
    f_idx]

eps = 1e-8
P = P / (P.sum() + eps)                # normalise to sum = 1  (PMF)

pred_time = P.sum(dim=1)               # (N,)  time marginal
pred_freq = P.sum(dim=0)               # (N,)  freq marginal

marginal_loss = (
    F.mse_loss(pred_time, time_energy) +
    F.mse_loss(pred_freq, freq_energy)
)

marginal_loss.backward()
optimizer.step()
```

Listing 10: Training step

7.4 Logging and sanity checks

```python
if epoch % 200 == 0:
    with torch.no_grad():
        rel_err_t = (pred_time - time_energy).norm() / time_energy.
            norm()
        rel_err_f = (pred_freq - freq_energy).norm() / freq_energy.
            norm()
        mass      = P.sum().item()
    print(
        f"Epoch {epoch:4d} | "
        f"Loss: {marginal_loss.item():.2e} | "
        f"rel_err_t: {rel_err_t.item():.4f} | "
        f"rel_err_f: {rel_err_f.item():.4f} | "
        f"mass: {mass:.4f}"
    )
```

Listing 11: Logging (every 200 epochs)

```python
assert torch.isfinite(P).all(),              "NaN or Inf in P"
assert torch.isfinite(marginal_loss).all(),  "NaN or Inf in loss"
```

Listing 12: Sanity checks (every epoch)

8 Step 5 — Cohen-Class Reference Distribution

Warning

Do *not* substitute a spectrogram for the pWVD. Do *not* use SciPy. Implement everything in PyTorch as specified below.

8.1 5a — Discrete pseudo-WVD

The discrete Wigner–Ville Distribution is defined as:

$$W(t,f) = \sum_{\tau} x[t+\tau]\, x^*[t-\tau]\, e^{-j2\pi f\tau/N},$$

where τ ranges over $[-N/2,\, N/2)$.

```python
# Zero-pad to length 2N for clean boundary handling
x_padded = torch.zeros(2 * N, dtype=torch.complex64, device=device)
x_padded[:N] = x                                  # x is the complex
    signal from Step 1

half = N // 2
tau_range = torch.arange(-half, half)             # shape (N,)

WVD = torch.zeros(N, N, device=device)

for t in range(N):
    t_plus  = (t + tau_range) % (2 * N)       # modular index into
        x_padded
    t_minus = (t - tau_range) % (2 * N)

    lag_product = x_padded[t_plus] * x_padded[t_minus].conj()  #
        shape (N,)
```

```
    # FFT over lag axis = DFT in frequency
    row = torch.fft.fft(lag_product, norm="ortho")
    WVD[t, :] = row.real                          # imaginary part is
        numerical noise
```

Listing 13: Pseudo-WVD computation in PyTorch

8.2 5b — 2-D Gaussian smoothing (Cohen-class operation)

Convolving the WVD with a 2-D kernel $\phi(t, f)$ produces a general Cohen-class distribution:

$$C_\phi(t, f) = (W * \phi)(t, f).$$

With a Gaussian kernel, this yields the *Rihaczek–smoothed* family and reliably suppresses cross-terms.

```
kernel_size = 15        # must be odd
sigma_k     = 2.0       # smoothing width in pixels

# Build 1-D Gaussian analytically
centre = kernel_size // 2
idx    = torch.arange(kernel_size, dtype=torch.float32, device=
    device)
g1d    = torch.exp(-0.5 * ((idx - centre) / sigma_k) ** 2)
g1d    = g1d / g1d.sum()                          # normalise

# Outer product -> 2-D kernel
g2d = torch.outer(g1d, g1d)                       # (K, K)
g2d = g2d / g2d.sum()                             # ensure sum = 1

# Apply as convolution
kernel = g2d.unsqueeze(0).unsqueeze(0)            # (1, 1, K, K)
pWVD   = WVD.unsqueeze(0).unsqueeze(0)            # (1, 1, N, N)

pWVD = F.conv2d(pWVD, kernel,
                padding=kernel_size // 2)         # same-size output
pWVD = pWVD.squeeze(0).squeeze(0)                 # (N, N)
```

Listing 14: 2-D Gaussian smoothing

8.3 5c — Post-processing for positive energy density

```
pWVD = torch.clamp(pWVD, min=0.0)                 # enforce non-
    negativity
pWVD = pWVD / pWVD.sum()                          # normalise to PMF
```

Listing 15: pWVD post-processing

```
assert pWVD.min() >= 0.0, \
    "pWVD has negative values after clamp"
assert abs(pWVD.sum().item() - 1.0) < 1e-5, \
    f"pWVD does not sum to 1 (got {pWVD.sum().item():.6f})"
```

Listing 16: pWVD validation

Expected result: a smooth diagonal ridge tracking the instantaneous frequency law $f_{\text{inst}}(t) = kt$ of the chirp, running from lower-left to upper-right in the (t, f) plane.

9 Step 6 — Visualisation

Produce a **single figure** with **three horizontal panels**, saved to `marginal_experiment_result.png`. Do **not** call `plt.show()`.

9.1 Figure skeleton

```
import matplotlib.pyplot as plt
import numpy as np

fig = plt.figure(figsize=(18, 5))
```

Listing 17: Figure setup

9.2 Panel 1 — Marginal comparison (two stacked line plots)

```
ax_t = fig.add_subplot(1, 3, 1)        # reuse as parent for two subplots
# better: use gridspec or subplot2grid

# Top: time marginals
ax_t1 = fig.add_subplot(3, 3, 1)
ax_t1.plot(t_grid.cpu(), time_energy.cpu(), color="steelblue",
           linewidth=1.5, label="True")
ax_t1.plot(t_grid.cpu(), pred_time.detach().cpu(), "--",
           color="firebrick", linewidth=1.5, label="Predicted")
ax_t1.set_title(f"Time marginal  (rel err = {rel_err_t:.4f})",
    fontsize=9)
ax_t1.legend(fontsize=8); ax_t1.set_xlabel("t")

# Bottom: freq marginals
ax_t2 = fig.add_subplot(3, 3, 4)
ax_t2.plot(f_norm.cpu(), freq_energy.cpu(), color="steelblue",
           linewidth=1.5, label="True")
ax_t2.plot(f_norm.cpu(), pred_freq.detach().cpu(), "--",
           color="firebrick", linewidth=1.5, label="Predicted")
ax_t2.set_title(f"Freq marginal  (rel err = {rel_err_f:.4f})",
    fontsize=9)
ax_t2.legend(fontsize=8); ax_t2.set_xlabel("f (normalised)")
```

Listing 18: Panel 1

Adapt the subplot indices to whatever layout strategy you use (`GridSpec` is recommended for clean alignment).

9.3 Panel 2 — Learned $P(t, f)$

```
P_np = P.detach().cpu().numpy()
vmin2 = np.percentile(P_np, 1)
vmax2 = np.percentile(P_np, 99)

ax2 = fig.add_subplot(1, 3, 2)
im2 = ax2.imshow(P_np.T,               # transpose so t on x-axis, f on y-axis
```

```
                  origin="lower", aspect="auto",
                  cmap="viridis", vmin=vmin2, vmax=vmax2)
ax2.set_xlabel("Time index");  ax2.set_ylabel("Frequency index")
ax2.set_title("Learned P(t,f)      Marginals Only")
plt.colorbar(im2, ax=ax2, label="Energy density")
```

Listing 19: Panel 2 — Learned distribution

Expected artefact: horizontal/vertical smear, separable slab, or uniform haze. There should be *no* diagonal ridge.

9.4 Panel 3 — Cohen-class reference (smoothed pWVD)

```
pWVD_np = pWVD.cpu().numpy()
vmin3 = np.percentile(pWVD_np, 1)
vmax3 = np.percentile(pWVD_np, 99)

ax3 = fig.add_subplot(1, 3, 3)
im3 = ax3.imshow(pWVD_np.T,
                 origin="lower", aspect="auto",
                 cmap="viridis", vmin=vmin3, vmax=vmax3)
ax3.set_xlabel("Time index");  ax3.set_ylabel("Frequency index")
ax3.set_title("Cohen-Class Reference  (smoothed pWVD)")
plt.colorbar(im3, ax=ax3, label="Energy density")
```

Listing 20: Panel 3 — Cohen-class reference

Expected structure: clear smooth diagonal ridge from lower-left to upper-right.

9.5 Save

```
plt.tight_layout()
plt.savefig("marginal_experiment_result.png",
            dpi=150, bbox_inches="tight")
plt.close(fig)
print("Figure saved to marginal_experiment_result.png")
```

9.6 Failure summary (print to stdout)

Listing 21: Required stdout output

```
========== FAILURE SUMMARY ==========
Marginal loss      : {marginal_loss:.6e}
Rel error (time)   : {rel_err_t:.6f}
Rel error (freq)   : {rel_err_f:.6f}
Mass of learned P  : {mass:.6f}

Diagnosis: Despite near-zero marginal errors, the learned P(t,f)
does not recover the diagonal ridge structure of the Cohen-class
reference. This confirms that 1D marginal constraints are
   insufficient
to uniquely determine a 2D energy distribution (ill-posed inverse
problem). Additional priors are required.
=====================================
```

10 Technical Constraints

- All computation (signal, FFT, pWVD, training, marginals) must use **PyTorch**. NumPy is permitted *only* for matplotlib interop (`.cpu().numpy()` calls).
- **No SciPy.** No `torchaudio`. No external signal processing libraries.
- Use `torch.fft.fft` — not the legacy `torch.rfft`.
- All tensors must be on `device` before training begins.
- Use these **exact variable names** throughout:
  `t_grid`, `f_grid`, `coords`, `P`, `pred_time`, `pred_freq`, `marginal_loss`, `pWVD`.
- Save the figure to `marginal_experiment_result.png` in the working directory. Do *not* call `plt.show()`.
- The script must complete on CPU in under 5 minutes.
- Supports `cuda`, `mps` (Apple Silicon), and `cpu` via the device selection block in Step 4.

11 Entry Point

Wrap all logic inside `main()`. No code should execute at module import time.

```python
def main():
    # Steps 1–6 here
    pass

if __name__ == "__main__":
    main()
```

Listing 22: Entry point

12 Mathematical Reference

This section provides the key equations for reference during implementation.

12.1 Windowed chirp

$$x(t) = \exp\left(-\frac{(t-t_0)^2}{2\sigma^2}\right)\exp\left(j\pi k t^2\right), \qquad t \in \left[-\tfrac{1}{2}, \tfrac{1}{2}\right).$$

Instantaneous frequency: $f_{\text{inst}}(t) = kt$.

12.2 Marginal constraints

$$\sum_f P(t,f) = |x(t)|^2, \qquad \sum_t P(t,f) = |X(f)|^2.$$

12.3 Discrete Wigner–Ville Distribution

$$W[t,f] = \sum_{\tau=-N/2}^{N/2-1} x[t+\tau]\, x^*[t-\tau]\, e^{-j2\pi f\tau/N}.$$

12.4 Cohen-class smoothing

$$C_\phi[t,f] = \sum_{t'} \sum_{f'} W[t',f']\, \phi[t-t',\, f-f'],$$

where ϕ is a 2-D Gaussian kernel $\phi[m,n] \propto \exp\!\left(-\frac{m^2+n^2}{2\sigma_\phi^2}\right)$.

12.5 Marginal loss

$$\mathcal{L}_{\text{marginal}} = \frac{1}{N}\sum_{t=1}^{N}\left(\hat{p}_t - p_t\right)^2 + \frac{1}{N}\sum_{f=1}^{N}\left(\hat{q}_f - q_f\right)^2,$$

where $\hat{p}_t = \sum_f P_\theta(t,f)$ and $p_t = |x(t)|^2 / \sum |x|^2$.

12.6 Relative marginal errors

$$\epsilon_t = \frac{\|\hat{\mathbf{p}} - \mathbf{p}\|_2}{\|\mathbf{p}\|_2}, \qquad \epsilon_f = \frac{\|\hat{\mathbf{q}} - \mathbf{q}\|_2}{\|\mathbf{q}\|_2}.$$

13 Definition of Done

Definition of Done

The script is complete when **all five** of the following hold:

1. Runs without errors on CPU, MPS, and CUDA.
2. All Parseval and normalisation assertions pass silently.
3. `marginal_loss` $< 10^{-4}$ by epoch 2000; relative marginal errors $\epsilon_t, \epsilon_f < 0.05$.
4. `marginal_experiment_result.png` is saved and clearly shows:
   - Panel 2 (learned P): **no** diagonal ridge visible — smear, separable slab, or uniform haze is expected.
   - Panel 3 (pWVD reference): **clear** smooth diagonal ridge from lower-left to upper-right.
5. Failure summary is printed to `stdout` with correct numeric values substituted.

14 Follow-up Scripts (Context Only)

This baseline motivates the following extensions, each in its own script. Do *not* implement them here.

Script	Prior / Constraint Added
`entropy_experiment.py`	Maximum-entropy regulariser $-\sum P \log P$ to encourage spread
`smoothness_experiment.py`	TV or Laplacian penalty on P
`radon_experiment.py`	Fractional Fourier / Radon projections at multiple angles
`moment_experiment.py`	First and second moment matching (mean IF, bandwidth)
`roi_experiment.py`	Region-of-interest conditioning from a prior chirp-rate estimate

Document generated for agent ingestion. Compile with `pdflatex`. All code blocks are copy-pasteable. Variable names and axis conventions are fixed; do not alter them.